



Lab Activity

Week 6: Advanced String Operations

Duration: 60 minutes

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Split strings into lists using the `split()` method
- Join lists into formatted strings using `join()`
- Search for patterns in text with `in`, `find()`, and `count()`
- Format strings professionally with `format()` and f-strings
- Parse and process real-world text data

Getting Started

1. Extract the given [week6_lab.zip](#) file to your desktop
2. Remember: Strings are immutable! Split and join create NEW strings
3. Keep the handout nearby for reference

1 Exercise 1: Split and Join Basics (10 minutes)

Let's master the fundamental string split and join operations!

Understanding `split()` and `join()`

Open [week6_ex1.py](#) and complete the code:

```
1 # Split and join exploration
2 sentence = "Python is really fun"
3
4 # Split by spaces (default)
5 words = sentence.____() # Fill in the method
6 print("Words:", words)
7 print("Type:", type(words)) # What type is it?
8
9 # Split by a specific character
10 data = "apple,banana,orange,grape"
11 fruits = data.____(',') # Split by comma
12 print("Fruits:", fruits)
13
14 # Join words back together
15 space_joined = ' '.____(____) # Join with spaces
16 dash_joined = '-'.____(____) # Join with dashes
17 print("Space joined:", space_joined)
```

```
18 print("Dash joined:", dash_joined)
```

Practice Tasks:

1. What does `split()` return? _____
2. Complete this: Make a sentence from the fruits list with " and " between items:

```
1 nice_list = _____._____ (____)
2 print(nice_list) # Should print: apple and banana and orange and grape
```

 **Tip:** Remember: The separator comes BEFORE `.join()` - think "separator.join(list)"

 **Checkpoint:** Split and join working? Show your teacher!

2 Exercise 2: CSV Data Parser (10 minutes)

Real-world data often comes in CSV format. Let's parse it!

Open `week6_ex2.py` and complete:

```
1 # Parse student grade data
2 print("== Grade Data Parser ==")
3
4 # CSV format: Name,Math,Science,English
5 grade_data = "Ali,85,92,78|Sara,90,88,95|Ahmed,78,85,82"
6
7 # Step 1: Split into student records
8 students = grade_data._____('|') # Split by |
9 print(f"Found {len(students)} students")
10
11 # Step 2: Process each student
12 for student_info in students:
13     # Split into fields
14     fields = student_info._____ (____) # Split by comma
15
16     name = fields[____] # Get name
17     math = int(fields[____]) # Get math grade
18     science = int(fields[____])
19     english = int(fields[____])
20
21     # Calculate average
22     average = (_____ + _____ + _____) / 3
23
24     print(f"{name}: Math={math}, Science={science}, English={english}, Avg={
        average:.1f}")
```

Extension Challenge:

```
1 # Find student with highest average
2 # Add your code here to track the best student
```

3 Exercise 3: Text Cleaning and Formatting (8 minutes)

Clean up messy text using split and join!

Open [week6_ex3.py](#) and complete:

```

1  # Text cleaning program
2  messy_text = "Too      many      spaces      here"
3  print("Original:", repr(messy_text))
4
5  # Clean up extra spaces
6  words = messy_text._____( ) # Split removes all whitespace
7  clean_text = _____._____(____) # Join with single spaces
8  print("Cleaned:", clean_text)
9
10 # Format user input
11 print("\n=== Name Formatter ===")
12 full_name = input("Enter your full name: ")
13
14 # Convert to title case with proper spacing
15 words = full_name.strip()._____( ) # Split into words
16 proper_name = ' '.join([word._____( ) for word in words])
17 print(f"Formatted: {proper_name}")

```

Practice: What will this output?

```

1  text = "  hello  world  "
2  result = '-'.join(text.split())

```

Answer: _____

 **Checkpoint:** Text cleaning working properly?

4 Exercise 4: Pattern Searching (10 minutes)

Master the in operator, find(), and count() for searching text!

Open [week6_ex4.py](#) and complete:

```

1  # Email validator
2  print("=== Email Validator ===")
3  email = input("Enter email: ").lower()
4
5  # Check for @ symbol
6  if '@' in email: # Use 'in' operator
7      print("Has @ symbol")
8
9      # Find position of @
10     at_pos = email.find('@')
11     print(f"@ is at position {at_pos}")
12
13     # Extract username and domain
14     username = email[:at_pos] # Before @
15     domain = email[at_pos+1:] # After @
16

```

```

17     print(f"Username: {username}")
18     print(f"Domain: {domain}")
19
20     # Check domain validity
21     if '.com' ___ domain or '.pk' ___ domain:
22         print("Valid domain")
23     else:
24         print("Invalid domain")
25 else:
26     print("Missing @ symbol")
27
28 # Count vowels in username
29 vowels = "aeiou"
30 vowel_count = 0
31 for char in username:
32     vowel_count += username.____(____) # Use count()
33 print(f"Vowels in username: {vowel_count}")

```

💡 **Tip:** find() returns -1 if not found. Use 'in' for simple yes/no checks!

5 Exercise 5: String Formatting Mastery (12 minutes)

Create professional-looking formatted output!

Open [week6_ex5.py](#) and complete:

```

1  # Grade report generator
2  print("=== Grade Report Card ===")
3
4  # Student data
5  name = "Ali Ahmed"
6  student_id = 12345
7  math = 85
8  science = 92
9  english = 78
10
11 # Format with .format()
12 header = "{:~40}".____(____) # Center the header
13 print(header)
14 print("=" * 40)
15
16 # Create aligned table
17 print("{:<15} | {:>10}".format("Subject", "Grade"))
18 print("-" * 28)
19 print("{:<15} | {:>10}".format(____, ____))
20 print("{:<15} | {:>10}".format(____, ____))
21 print("{:<15} | {:>10}".format(____, ____))
22 print("-" * 28)
23
24 # Calculate and display average
25 average = (math + science + english) / 3
26 print("{:<15} | {:>10.1f}".format("Average", average))
27
28 # Using f-strings (Python 3.6+)
29 print(f"\nStudent: {name} (ID: {student_id})")
30 print(f"Status: {'PASS' if average >= 60 else 'FAIL'}")
31
32 # Money formatting

```

```

33 price = 1234.5
34 print(f"\nPrice: ${price:.2f}") # 2 decimal places
35 print(f"With comma: ${price:,.2f}") # Comma separator

```

Practice: Complete the format specifiers

- Left align in 20 chars: { :----- }
- Right align in 10 chars: { :----- }
- Center in 15 chars: { :----- }
- Float with 3 decimals: { :----- }

✔ **Checkpoint:** Report card formatting correctly? Excellent!

6 Exercise 6: Word Analyzer Program (10 minutes)

Combine all string operations in one program!

Open [week6_ex6.py](#) and complete:

```

1  # Comprehensive text analyzer
2  print("=== TEXT ANALYZER ===")
3  text = input("Enter a sentence: ")
4
5  # Basic statistics
6  word_count = len(text.____())
7  char_count = len(text)
8  char_no_spaces = len(text.replace(' ', ''))
9
10 print("\n" + "=" * 40)
11 print(f"{'Statistic':<25} | {'Value':>10}")
12 print("=" * 40)
13 print(f"{'Words':<25} | {word_count:>10}")
14 print(f"{'Characters (with spaces)':<25} | {char_count:>10}")
15 print(f"{'Characters (no spaces)':<25} | {char_no_spaces:>10}")
16
17 # Find and count specific word
18 search_word = input("\nSearch for word: ").lower()
19 if search_word ___ text.lower():
20     position = text.lower().____(____)
21     occurrences = text.lower().____(____)
22     print(f"Found '{search_word}' at position {position}")
23     print(f"    Appears {occurrences} time(s)")
24 else:
25     print(f"'{search_word}' not found")
26
27 # Most common letter
28 letter_counts = {}
29 for char in text.lower():
30     if char.isalpha(): # Only count letters
31         if char in letter_counts:
32             letter_counts[char] += 1
33         else:

```

```

34         letter_counts[char] = 1
35
36 if letter_counts:
37     most_common = max(letter_counts, key=letter_counts.get)
38     print(f"\nMost common letter: '{most_common}' ({letter_counts[
39         most_common]} times)")
40
41 # Word length analysis
42 words = text.split()
43 if words:
44     longest = max(words, key=len)
45     shortest = min(words, key=len)
46     print(f"Longest word: {longest} ({len(longest)} chars)")
47     print(f"Shortest word: {shortest} ({len(shortest)} chars)")

```

✓ **Checkpoint:** Analyzer working with all features? Amazing!

7 Lab Summary

✓ **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Used `split()` to break strings into lists
- ☐ Used `join()` to combine lists into strings
- ☐ Parsed CSV-style data successfully
- ☐ Cleaned text by removing extra spaces
- ☐ Searched for patterns with `in`, `find()`, and `count()`
- ☐ Created formatted output with proper alignment
- ☐ Built a complete text analyzer
- ☐ Worked effectively with your partner

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	☹️ Need Help	😊 Getting It	😄 Got It!
<code>split()</code> and <code>join()</code>	☐	☐	☐
Pattern searching (<code>in</code> , <code>find</code> , <code>count</code>)	☐	☐	☐
String formatting (<code>format()</code> , f-strings)	☐	☐	☐
Parsing real-world data	☐	☐	☐

Which string operation will be most useful for your future projects?

What was the trickiest part about string formatting?

😊 Excellent work processing text like a pro!